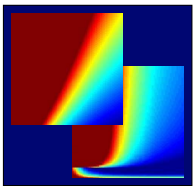
Om Sanan

CS/CNS/EE 156a Learning Systems
Caltech - Fall 2025
https://caltech.instructure.com/courses/8882
(Learning From Data campus version)



Homework # 5

Due Monday, November 3, 2025, at 2:00 PM PST

*Definitions and notation follow the lectures. All questions have multiple-choice answers (***[a], [b], [c], ...***). Collaboration is allowed but without discussing selected or excluded choices. Your solutions must be based on your own work. See the initial "***Course Description and Policies***" handout for important details about collaboration, open book, and chatGPT policies.*

Note about the homework

- Answer each question by deriving the answer (carries 6 points) then selecting from the multiple-choice answers (carries 4 points). You can select 1 or 2 of the multiple-choice answers for each question, but you will get 4 or 2 points, respectively, for a correct answer. See the initial "**Course Description and Policies**" handout for important details.
- The problems range from easy to difficult, and from practical to theoretical. Some problems require running a full experiment to arrive at the answer.
- The answer may not be obvious or numerically close to one of the choices, but one (and only one) choice will be correct if you follow the instructions precisely in each problem. You are encouraged to explore the problem further by experimenting with variations on these instructions, for the learning benefit.
- You are encouraged to take part in the Piazza discussion forum. Please make sure you don't discuss specific answers, or specific excluded answers, before the homework is due.

© 2012-2025 Yaser Abu-Mostafa. All rights reserved. No redistribution in any format. No translation or derivative products without written permission.

• Linear Regression Error

Consider a noisy target $y = \mathbf{w}^{*T}\mathbf{x} + \epsilon$, where $\mathbf{x} \in \mathbb{R}^d$ (with the added coordinate $x_0 = 1$), $y \in \mathbb{R}$, $\mathbf{w}^*$ is an unknown vector, and ϵ is a noise term with zero mean and σ^2 variance. Assume ϵ is independent of $\mathbf{x}$ and of all other ϵ's. If linear regression is carried out using a training data set $\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)\}$, and outputs the parameter vector $\mathbf{w}_{\text{lin}}$, it can be shown that the expected in-sample error E_{in} with respect to $\mathcal{D}$ is given by:

$$\mathbb{E}_{\mathcal{D}}[E_{\text{in}}(\mathbf{w}_{\text{lin}})] = \sigma^2 \left(1 - \frac{d+1}{N}\right)$$

1. For $\sigma = 0.1$ and $d = 8$, which among the following choices is the smallest number of examples N that will result in an expected E_{in} greater than 0.008?

[a] 10

[b] 25

[c] 100

[d] 500

[e] 1000

0.01(1-9/N) > 0.008

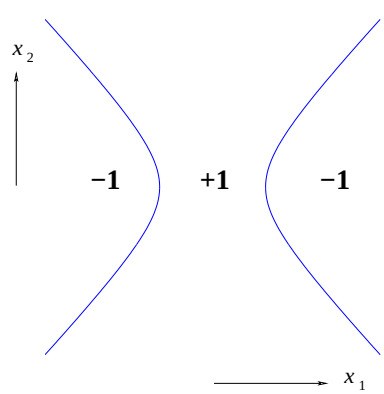
N > 45 (using CAS)

• Nonlinear Transforms

In linear classification, consider the feature transform $\Phi : \mathbb{R}^2 \to \mathbb{R}^2$ (plus the added zeroth coordinate) given by:

$$\Phi(1, x_1, x_2) = (1, x_1^2, x_2^2)$$

2. Which of the following sets of constraints on the weights in the $\mathcal{Z}$ space could correspond to the hyperbolic decision boundary in $\mathcal{X}$ depicted in the figure?



You may assume that $\tilde{w}_0$ can be selected to achieve the desired boundary.

[a] $\tilde{w}_1 = 0, \tilde{w}_2 > 0$

[b] $\tilde{w}_1 > 0, \tilde{w}_2 = 0$

[c] $\tilde{w}_1 > 0, \tilde{w}_2 > 0$

[d] $\tilde{w}_1 < 0, \tilde{w}_2 > 0$

[e] $\tilde{w}_1 > 0, \tilde{w}_2 < 0$

w_0 + w_1*x_1^2 + w_2*x_2^2 = 0

For hyperbola: need opposite signs on x_1^2 and x_2^2 terms

w_1*x_1^2 + w_2*x_2^2 = w_0 > 0

we want x_1^2 term to be positive and x_2^2 term to be negative

for parabola to open left/right

this yields +1 class at center, -1 at sides

Now, consider the 4th order polynomial transform from the input space $\mathbb{R}^2$:

$$\Phi_4 : \mathbf{x} \to (1, x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, x_1^2x_2, x_1x_2^2, x_2^3, x_1^4, x_1^3x_2, x_1^2x_2^2, x_1x_2^3, x_2^4)$$

3. What is the smallest value among the following choices that is *not* smaller than the VC dimension of a linear model in this transformed space?

[a] 3

[b] 5

[c] 15

[d] 20

[e] 21

15 features is the length of the map shown

This also equals C(2+4,4)=15

so VC dim is 15

• Gradient Descent

Consider the nonlinear error surface $E(u, v) = (ue^v - 2ve^{-u})^2$. We start at the point $(u, v) = (1, 1)$ and minimize this error using gradient descent in the uv space. Use $\eta = 0.1$ (learning rate, not step size).

4. What is the partial derivative of $E(u, v)$ with respect to u, i.e., $\frac{\partial E}{\partial u}$?

[a] $(ue^v - 2ve^{-u})^2$

[b] $2(ue^v - 2ve^{-u})$

[c] $2(e^v + 2ve^{-u})$

[d] $2(e^v - 2ve^{-u})(ue^v - 2ve^{-u})$

[e] $2(e^v + 2ve^{-u})(ue^v - 2ve^{-u})$

Let f(u, v) = u e^v − 2v e^(−u)

Then E = f^2

∂E/∂u = 2f · ∂f/∂u

∂/∂u(u e^v) = e^v

∂/∂u(−2v e^(−u)) = −2v · (−e^(−u)) = 2v e^(−u)

So ∂f/∂u = e^v + 2v e^(−u)

∂E/∂u = 2(e^v + 2v e^(−u))(u e^v − 2v e^(−u))

5. How many iterations (among the given choices) does it take for the error $E(u, v)$ to fall below 10^{-14} for the first time? In your programs, make sure to use double precision to get the needed accuracy.

[a] 1

```
Error fell below 1e-14 after 10 iterations
see code at bottom of document
```

[b] 3

[c] 5

[d] 10

[e] 17

6. After running enough iterations such that the error has just dropped below 10^{-14}, what are the closest values (in Euclidean distance) among the following choices to the final (u, v) you got in Problem 5?

[a] $(1.000, 1.000)$

[b] $(0.713, 0.045)$

```
u=0.044736, v=0.023959
```

[c] $(0.016, 0.112)$

[d] $(-0.083, 0.029)$

[e] $(0.045, 0.024)$

7. Now, we will compare the performance of "coordinate descent." In each iteration, we have two steps along the 2 coordinates. Step 1 is to move only along the u coordinate to reduce the error (assume first-order approximation holds like in gradient descent), and step 2 is to reevaluate and move only along the v coordinate to reduce the error (again, assume first-order approximation holds). Use the same learning rate of $\eta = 0.1$ as we did in gradient descent. What will the error $E(u, v)$ be closest to after 15 full iterations (30 steps)?

[a] 10^{-1}

```
Final error after 15 iterations: 0.139814
see code below
```

[b] 10^{-7}

[c] 10^{-14}

[d] 10^{-17}

[e] 10^{-20}

• Logistic Regression

In this problem you will create your own target function f (probability in this case) and data set $\mathcal{D}$ to see how Logistic Regression works. For simplicity, we will take f to be a 0/1 probability so y is a deterministic function of $\mathbf{x}$.

Take $d = 2$ so you can visualize the problem, and let $\mathcal{X} = [-1, 1] \times [-1, 1]$ with uniform probability of picking each $\mathbf{x} \in \mathcal{X}$. Choose a line in the plane as the boundary between $f(\mathbf{x}) = 1$ (where y has to be $+1$) and $f(\mathbf{x}) = 0$ (where y has to be -1) by taking two random, uniformly distributed points from $\mathcal{X}$ and taking the line passing through

them as the boundary between $y = \pm 1$. Pick $N = 100$ training points at random from $\mathcal{X}$, and evaluate the outputs y_n for each of these points $\mathbf{x}_n$.

Run Logistic Regression with Stochastic Gradient Descent to find g, and estimate E_{out} (the **cross entropy** error) by generating a sufficiently large, separate set of points to evaluate the error. Repeat the experiment for 100 runs with different targets and take the average. Initialize the weight vector of Logistic Regression to all zeros in each run. Stop the algorithm when $\|\mathbf{w}^{(t-1)} - \mathbf{w}^{(t)}\| < 0.01$, where $\mathbf{w}^{(t)}$ denotes the weight vector at the end of epoch t. An epoch is a full pass through the N data points (use a random permutation of $1, 2, \cdots, N$ to present the data points to the algorithm within each epoch, and use different permutations for different epochs). Use a learning rate of 0.01.

8. Which of the following is closest to E_{out} for $N = 100$?

[a] 0.025
[b] 0.050
[c] 0.075
[d] 0.100
[e] 0.125

Average E_out: 0.099947
see code below

9. How many epochs does it take on average for Logistic Regression to converge for $N = 100$ using the above initialization and termination rules and the specified learning rate? Pick the value that is closest to your results.

[a] 350
[b] 550
[c] 750
[d] 950
[e] 1750

Average epochs: 353.14
see code below

- **PLA as SGD**

10. The Perceptron Learning Algorithm can be implemented as SGD using which of the following error functions $\mathrm{e}_n(\mathbf{w})$? Ignore the points $\mathbf{w}$ at which $\mathrm{e}_n(\mathbf{w})$ is not twice differentiable.

[a] $\mathrm{e}_n(\mathbf{w}) = e^{-y_n \mathbf{w}^\intercal \mathbf{x}_n}$
[b] $\mathrm{e}_n(\mathbf{w}) = -y_n \mathbf{w}^\intercal \mathbf{x}_n$
[c] $\mathrm{e}_n(\mathbf{w}) = (y_n - \mathbf{w}^\intercal \mathbf{x}_n)^2$
[d] $\mathrm{e}_n(\mathbf{w}) = \ln(1 + e^{-y_n \mathbf{w}^\intercal \mathbf{x}_n})$
[e] $\mathrm{e}_n(\mathbf{w}) = -\min(0, y_n \mathbf{w}^\intercal \mathbf{x}_n)$

The PLA update rule is:
- If misclassified: w <- w + y_n*x_n
- If correct: no update
For SGD with update w <- w - eta*∇e_n(w), we need:
- Misclassified (y_n*w^T*x_n < 0): ∇e_n(w) = -y_n*x_n
- Correct (y_n*w^T*x_n > 0): ∇e_n(w) = 0
Checking (e): e_n(w) = -min(0, y_n*w^T*x_n) = max(0, -y_n*w^T*x_n)
- When y_n*w^T*x_n > 0 (correct): e_n= 0, so ∇e_n = 0
- When y_n*w^T*x_n < 0 (misclassified) e_n = -y_n*w^T*x_n, so ∇e_n = -y_n*xN
This matches PLA exactly :)

Problem 5

```
import numpy as np

# error
def E(u, v):
    term = u * np.exp(v) - 2 * v * np.exp(-u)
    return term ** 2

# compute gradient of E
def gradient(u, v):
    f = u * np.exp(v) - 2 * v * np.exp(-u)

    # dE/du
    # note these are really partials not d's
    df_du = np.exp(v) + 2 * v * np.exp(-u)
    dE_du = 2 * f * df_du

    # dE/dv
    df_dv = u * np.exp(v) - 2 * np.exp(-u)
    dE_dv = 2 * f * df_dv

    return dE_du, dE_dv

# params
u, v = 1.0, 1.0
eta = 0.1
threshold = 1e-14

print(f"Initial: u={round(u, 6)}, v={round(v, 6)}, E={round(E(u, v),
6)}")

iter = 0
while E(u, v) >= threshold:
    iter += 1

    # calculae gradient
    dE_du, dE_dv = gradient(u, v)

    # update using GD
    u = u - eta * dE_du
    v = v - eta * dE_dv

    error = E(u, v)
    print(f"Iteration {iter}: u={round(u, 6)}, v={round(v, 6)},
E={round(error, 6)}")

    if iter > 20:
```

```
        break

print(f"Error fell below {threshold} after {iter} iterations")
```

```
Initial: u=1.0, v=1.0, E=3.930397
Iteration 1: u=-0.369543, v=0.213921, E=1.15951
Iteration 2: u=0.030521, v=-0.507934, E=1.007407
Iteration 3: u=0.107523, v=-0.12221, E=0.099009
Iteration 4: u=0.065645, v=-0.015167, E=0.008661
Iteration 5: u=0.047841, v=0.01849, E=0.000182
Iteration 6: u=0.044999, v=0.023499, E=1e-06
Iteration 7: u=0.044756, v=0.023924, E=0.0
Iteration 8: u=0.044738, v=0.023956, E=0.0
Iteration 9: u=0.044736, v=0.023959, E=0.0
Iteration 10: u=0.044736, v=0.023959, E=0.0
Error fell below 1e-14 after 10 iterations
```

Problem 7

```
# coordinate descent - add partial derivative functions
def partial_u(u, v):
    f = u * np.exp(v) - 2 * v * np.exp(-u)
    df_du = np.exp(v) + 2 * v * np.exp(-u)
    return 2 * f * df_du

def partial_v(u, v):
    f = u * np.exp(v) - 2 * v * np.exp(-u)
    df_dv = u * np.exp(v) - 2 * np.exp(-u)
    return 2 * f * df_dv

# reset to initial point
u, v = 1.0, 1.0
eta = 0.1

print(f"Initial: u={round(u, 6)}, v={round(v, 6)}, E={round(E(u, v),
6)}")

# 15 iters = 30 steps
for iter in range(1, 16):
    # step 1: move along u only
    dE_du = partial_u(u, v)
    u = u - eta * dE_du

    # step 2: move along v only
    dE_dv = partial_v(u, v)
    v = v - eta * dE_dv

    error = E(u, v)
```

```
    print(f"Iteration {iter}: u={round(u, 6)}, v={round(v, 6)},
E={round(error, 6)}")

print(f"Final error after 15 iterations: {round(error, 6)}")
```

```
Initial: u=1.0, v=1.0, E=3.930397
Iteration 1: u=-0.369543, v=-2.039923, E=34.290163
Iteration 2: u=6.392474, v=-2.178699, E=0.534143
Iteration 3: u=6.376995, v=-2.283473, E=0.432661
Iteration 4: u=6.364607, v=-2.368214, E=0.36504
Iteration 5: u=6.354277, v=-2.439595, E=0.316468
Iteration 6: u=6.345421, v=-2.501375, E=0.279763
Iteration 7: u=6.337678, v=-2.5559, E=0.250986
Iteration 8: u=6.330806, v=-2.604735, E=0.227783
Iteration 9: u=6.324636, v=-2.648984, E=0.208657
Iteration 10: u=6.319042, v=-2.689452, E=0.192606
Iteration 11: u=6.313931, v=-2.726749, E=0.178935
Iteration 12: u=6.309231, v=-2.761345, E=0.167145
Iteration 13: u=6.304884, v=-2.793613, E=0.156869
Iteration 14: u=6.300845, v=-2.823852, E=0.14783
Iteration 15: u=6.297076, v=-2.852307, E=0.139814
Final error after 15 iterations: 0.139814
```

Problems 8-9

```
import numpy as np

# get random line in [-1,1]x[-1,1]
def generate_target_function():
    points = np.random.uniform(-1, 1, (2, 2))
    return points

# classify points based on which side of line they fall
def classify_points(X, line_points):
    p1, p2 = line_points
    # line: (y - y1) = m(x - x1) -> ax + by + c = 0
    # use cross product to determine side
    y = np.zeros(len(X))
    for i, x in enumerate(X):
        # vector from p1 to p2
        v = p2 - p1
        # vector from p1 to x
        w = x - p1
        # cross product (in 2D, just z-component)
        cross = v[0] * w[1] - v[1] * w[0]
        y[i] = 1 if cross > 0 else -1
    return y
```

```python
# sigmoid function
def sigmoid(s):
    return np.where(s >= 0, 1 / (1 + np.exp(-s)), np.exp(s) / (1 +
np.exp(s)))

# cross entropy error
def error(X, y, w):
    # bias term
    X_bias = np.c_[np.ones(len(X)),X]
    s = X_bias @ w
    # cross entropy: (1/N) * sum(ln(1 + e^(-y*s)))
    return np.mean(np.log(1+np.exp(-y*s)))

# logistic regression with SGD
def lr(X, y, eta=0.01, threshold=0.01):
    N = len(X)
    # bias term
    X_bias = np.c_[np.ones(N),X]

    # set weights to 0
    w = np.zeros(X_bias.shape[1])

    epochs = 0
    while True:
        w_old = w.copy()
        epochs += 1

        # random permutation for this epoch
        indices = np.random.permutation(N)

        # SGD update
        for idx in indices:
            x_n = X_bias[idx]
            y_n = y[idx]

            # gradient of cross-entropy for single point
            # -(y_n * x_n) / (1 + exp(y_n * w^T * x_n))
            s = w @ x_n
            gradient = -(y_n*x_n) / (1 + np.exp(y_n*s))

            w = w - eta * gradient

        # check convergence
        if np.linalg.norm(w - w_old) < threshold:
            break

    return w, epochs

# run experiment
np.random.seed(42)
```

```python
num_runs = 100
N_train = 100
N_test = 1000

e_out_list = []
epochs_list = []

for run in range(num_runs):
    # generate target function
    line_points = generate_target_function()

    # generate training data
    X_train = np.random.uniform(-1, 1, (N_train, 2))
    y_train = classify_points(X_train, line_points)

    # run logistic regression
    w, epochs = lr(X_train, y_train)

    # generate test data for E_out
    X_test = np.random.uniform(-1, 1, (N_test, 2))
    y_test = classify_points(X_test, line_points)

    # calculate E_out
    e_out = error(X_test, y_test, w)

    e_out_list.append(e_out)
    epochs_list.append(epochs)

    if (run+1)%10 == 0:
        print(f"Run {run+1}/100 complete")

avg_e_out = np.mean(e_out_list)
avg_epochs = np.mean(epochs_list)

print(f"Average E_out: {round(avg_e_out, 6)}")
print(f"Average epochs: {round(avg_epochs, 2)}")
```

```
Run 10/100 complete
Run 20/100 complete
Run 30/100 complete
Run 40/100 complete
Run 50/100 complete
Run 60/100 complete
Run 70/100 complete
Run 80/100 complete
Run 90/100 complete
Run 100/100 complete
Average E_out: 0.099947
Average epochs: 353.14
```